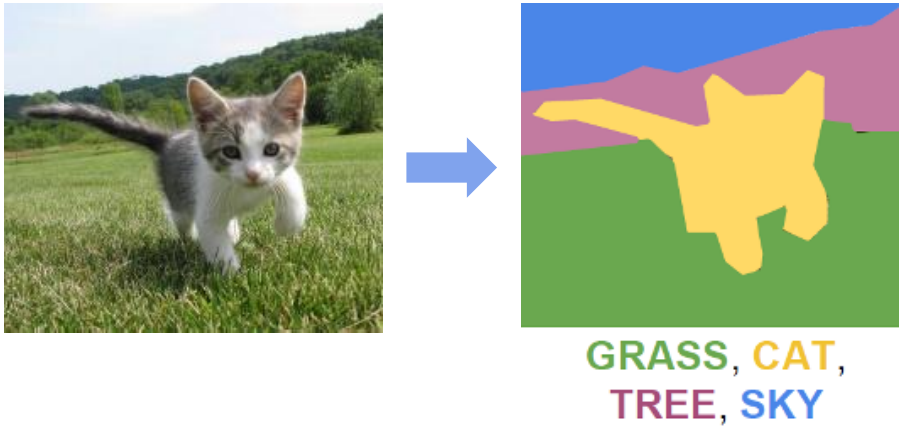
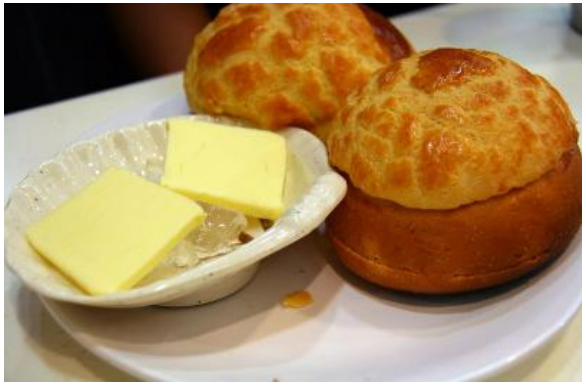
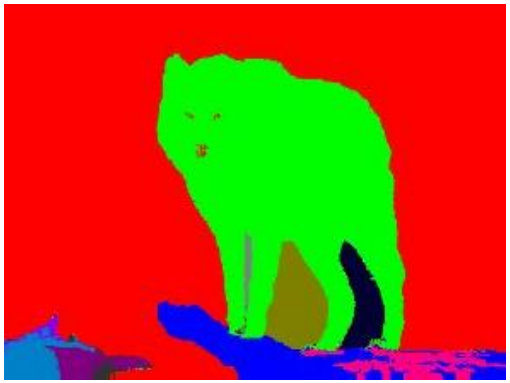
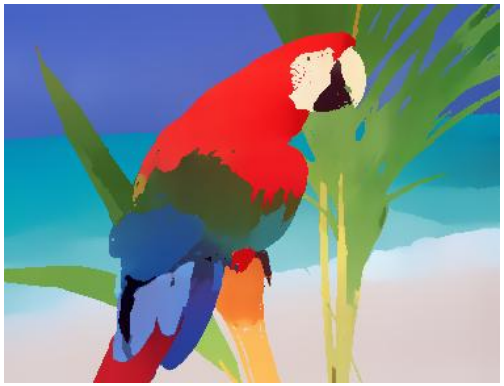
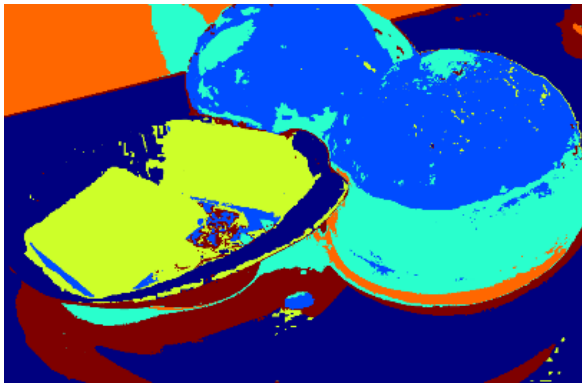
Deep Learning for Semantic Segmentation



Matteo Ferrara
matteo.ferrara@unibo.it

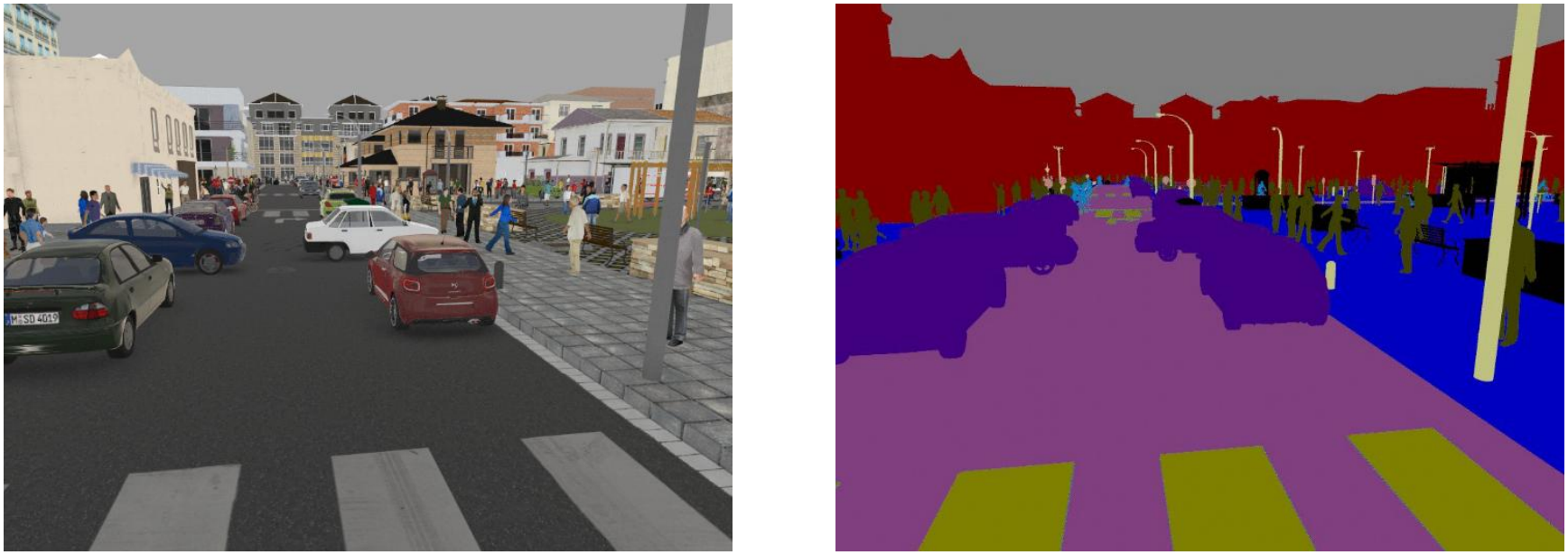
Segmentation

Con il termine *segmentation* ci si riferisce a un insieme di metodi il cui obbiettivo è partizionare un'immagine di input in regioni con proprietà comuni (es. colore, *edge*).

Semantic segmentation

L'obbiettivo degli approcci di *semantic segmentation* è la classificazione di ogni pixel di un'immagine sulla base di un insieme predefinito di classi.



Instance segmentation

Le tecniche di *instance segmentation* classificano ogni pixel di un'immagine identificando le diverse istanze delle varie classi.

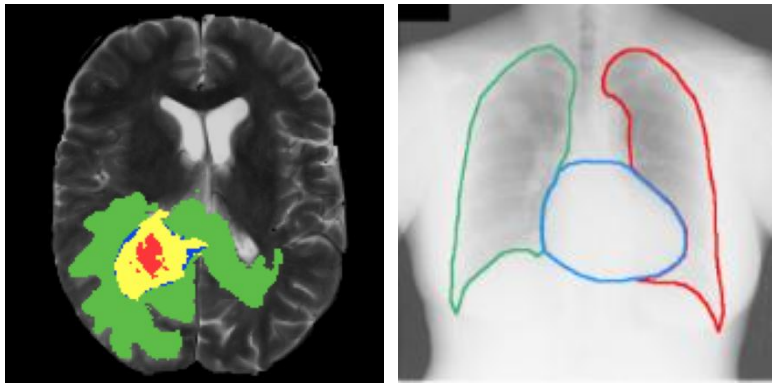
Semantic segmentation

Instance segmentation

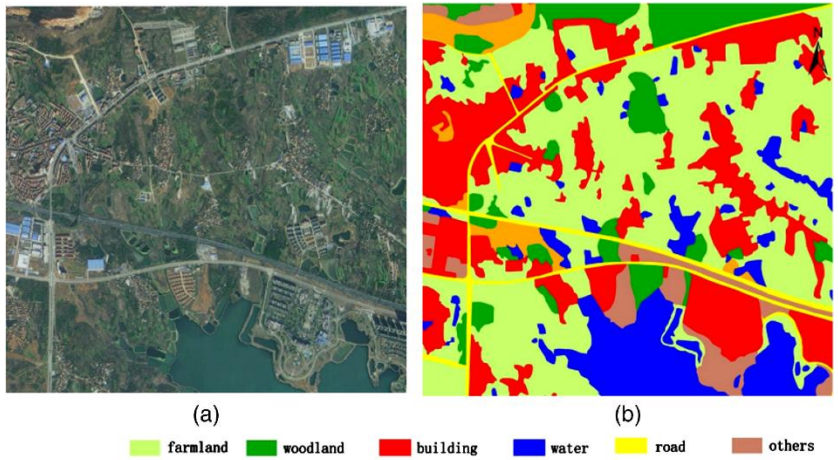
Ambiti applicativi

Vi sono numerosi ambiti applicativi in cui la semantic segmentation è molto utile tra cui:

Immagini medicali



Auto a guida autonoma



Analisi di immagini satellitari

Valutazione delle prestazioni

Input image | Ground truth | Predicted

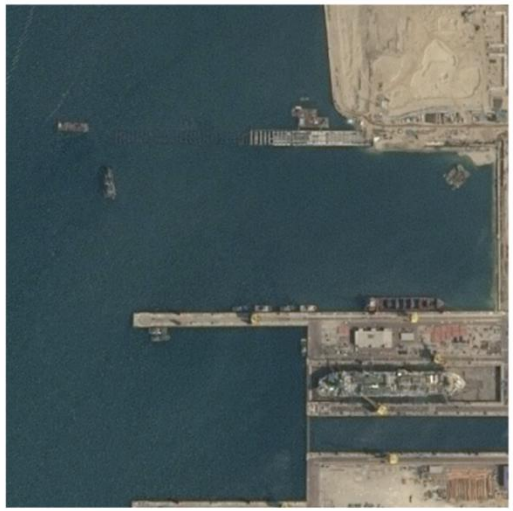
In letteratura esistono vari indicatori utilizzati per misurare le prestazioni di un sistema di segmentazione.

- Accuratezza di classificazione – la percentuale di pixel correttamente classificati.
- Errore di classificazione – è il complemento dell'accuratezza.

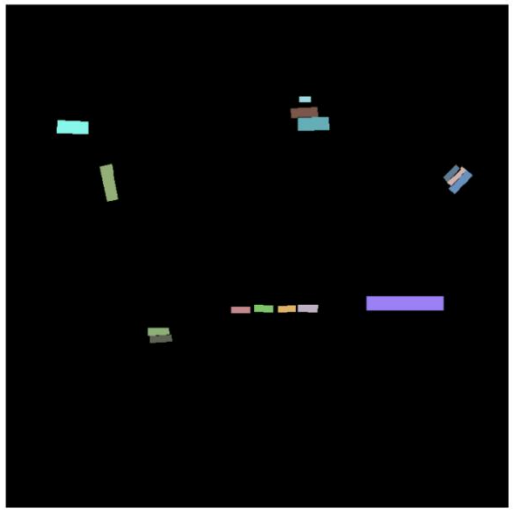
Valutazione delle prestazioni (2)

Attenzione ai problemi con cardinalità delle classi molto sbilanciata.

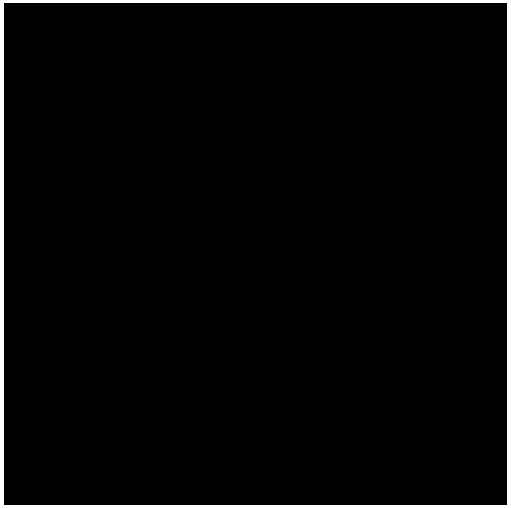
Input image



Ground truth



Predicted



Accuratezza del 95%

Valutazione delle prestazioni (3)

- *Confusion matrix* – ogni cella (i, j) riporta la percentuale di casi in cui il sistema ha assegnato la classe j a un pixel la cui classe reale è i.

		Predetti				
		Gatto	Cane	Coniglio	Capra	Maiale
Reali	Gatto	71%	29%	0%	0%	0%
	Cane	38%	38%	24%	0%	0%
	Coniglio	0%	8%	92%	0%	0%
	Capra	0%	11%	0%	83%	6%
	Maiale	0%	29%	0%	17%	54%

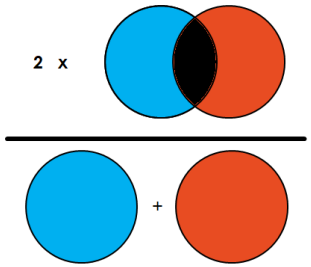
Valutazione delle prestazioni (4)

- *Intersection over Union* (IoU) o *Jaccard index* – quantifica la percentuale di *overlapping* tra la maschera restituita dal sistema e la maschera attesa.

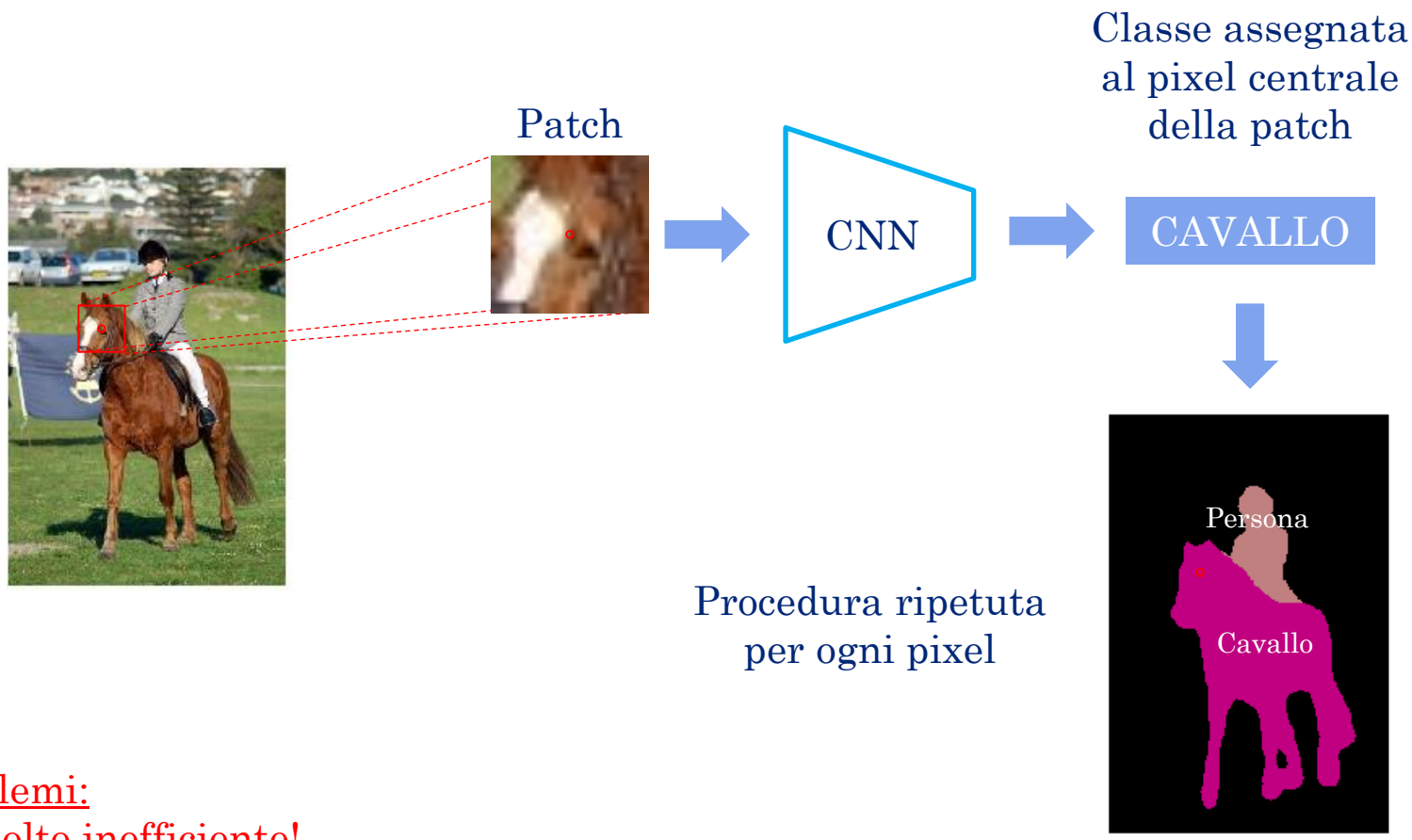
$$IoU = \frac{|At \cap Re|}{|At \cup Re|}$$

- *Dice index* (DSC) o F1-*score* – misura il rapporto tra i pixel in comune tra la maschera restituita e la maschera attesa e il numero di pixel presente in ogni maschera.

$$DSC = \frac{2 \cdot |At \cap Re|}{|At| + |Re|}$$



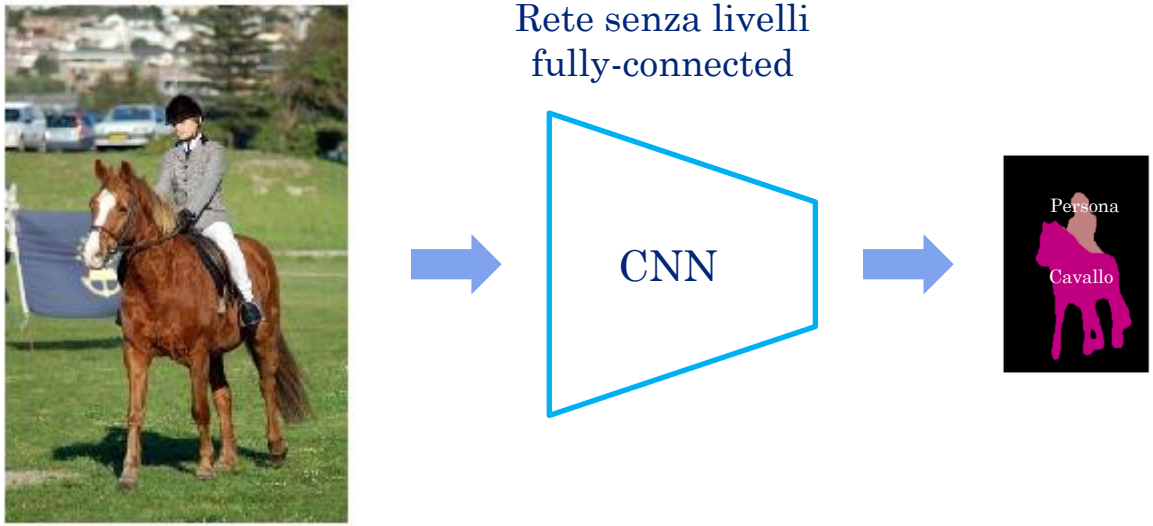
Semantic segmentation e Deep learning



Problemi:

- molto inefficiente!
- si utilizzano solo le informazioni contenute nell'intorno del pixel.

Semantic segmentation e Deep learning (2)

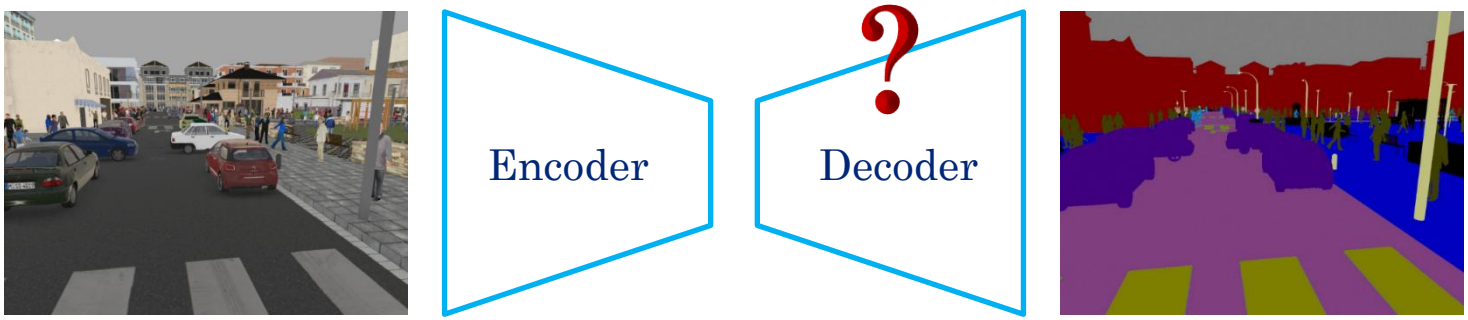


Problema: output più piccolo dell'input a causa della riduzione delle dimensioni spaziali.

Struttura Encoder - Decoder

In letteratura sono state proposte diverse reti *deep* per risolvere il problema della semantic segmentation. La maggior parte di esse condivide la medesima struttura generale - una rete di codifica (*encoder*) seguita da una rete di decodifica (*decoder*).

- Encoder: effettua un *mapping* a bassa risoluzione dell'immagine aggregando (da più livelli) *feature* altamente efficienti nel discriminare le varie classi.
- Decoder: ha il compito di proiettare «semanticamente» le *feature* discriminanti (bassa risoluzione) apprese dall'*encoder* sullo spazio originale dei pixel (alta risoluzione) per ottenere una classificazione «densa».

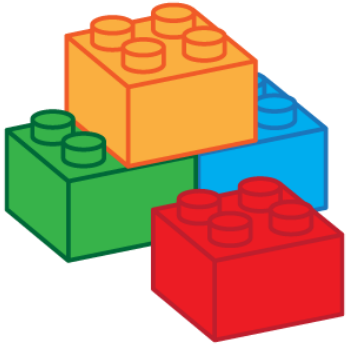


Struttura Encoder – Decoder (2)

Una rete decoder, per poter ottenere un output delle stesse dimensioni dell'immagine di input, deve essere in grado di «invertire» le operazioni di convoluzione e pooling utilizzate nella rete encoder.

Per fare ciò si avvale di due nuove operazioni:

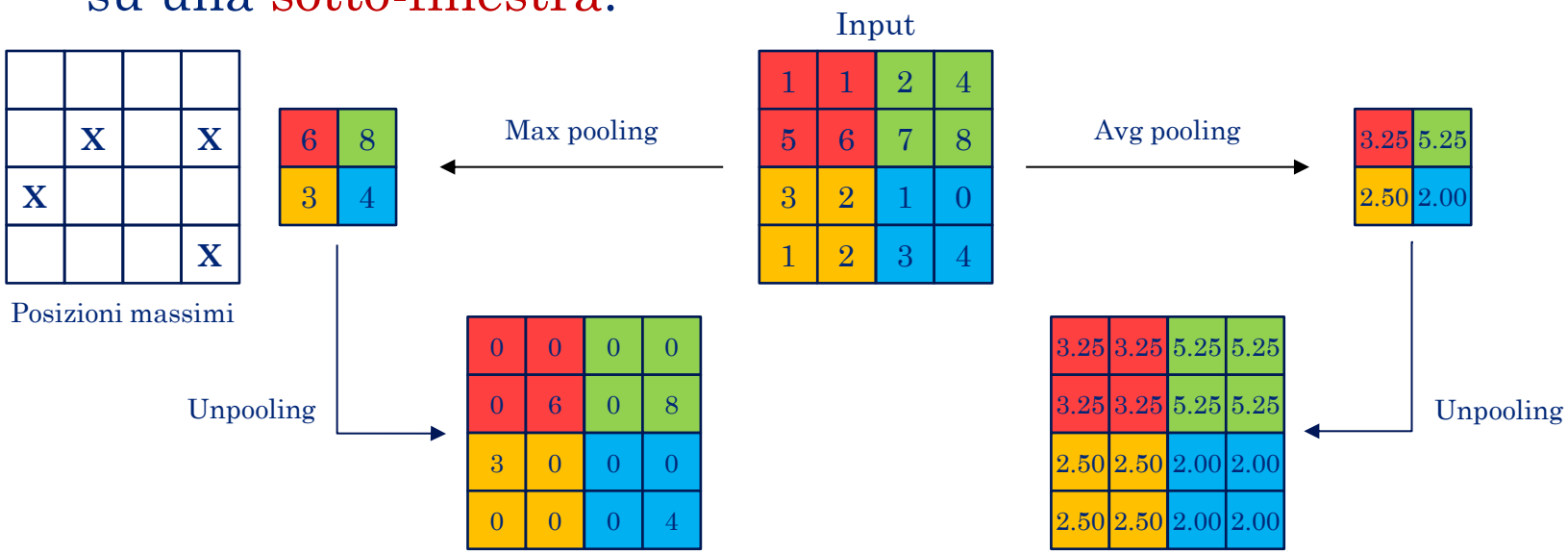
1) Unpooling
2) Transposed convolution



Unpooling

L'obbiettivo dell'operazione di *unpooling* è di invertire una precedente operazione di pooling aumentando le dimensioni della feature map di input.

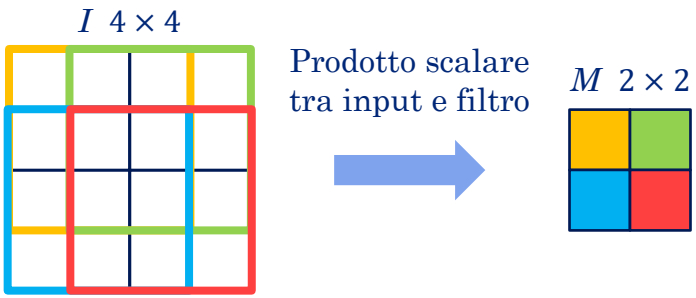
Mentre l'operazione di pooling riduce la risoluzione condensando una sotto-finestra in un singolo valore, viceversa l'unpooling amplia la risoluzione distribuendo un singolo valore su una sotto-finestra.



Transposed convolution

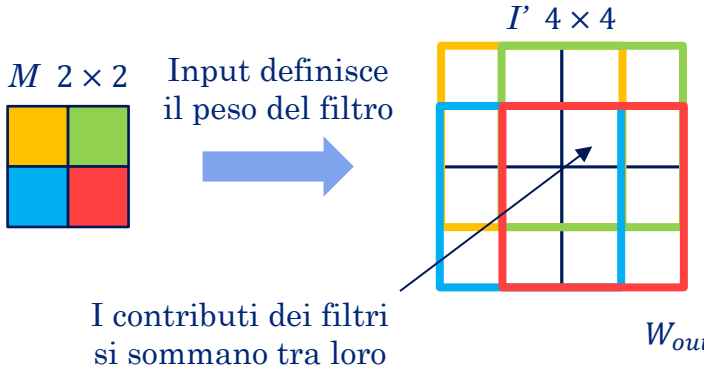
Data una feature map *M*, ottenuta applicando l'operazione di convoluzione a una feature map iniziale *I*, la *transposed convolution* è l'operazione che, applicata a *M*, consente di ottenere una feature map *I'* con le stesse dimensioni (*shape*) di *I*.

Convoluzione
K 3×3
Stride=1
Padding=0



$$W_{out} = \frac{(W_{in} - F + 2 \cdot Padding)}{Stride} + 1$$

Transposed convolution
K 3×3
Stride=1
Padding=0



$$W_{out} = (W_{in} - 1) \cdot Stride + F - 2 \cdot Padding$$

Transposed convolution (2)

Esempio

Input

0	1
2	3

Kernel

0	1	2
3	4	5
6	7	8

Output

Stride=1
Padding=0

0	0	0	
0	0	0	
0	0	0	

+

	0	1	2
	3	4	5
	6	7	8

+

0	2	4	
6	8	10	
12	14	16	

+

	0	3	6
	9	12	15
	18	21	24

=

0	0	1	2
0	5	11	11
6	23	29	23
12	32	37	24

Stride=1
Padding=1

=

	0	0	
	0	0	

+

	3	4	
	6	7	

+

	2	4	
	8	10	

+

	0	3	
	9	12	

=

	5	11	
	23	29	

Stride=2
Padding=0

0	0	0		
0	0	0		
0	0	0		

+

		0	1	2
		3	4	5
		6	7	8

+

0	2	4		
6	8	10		
12	14	16		

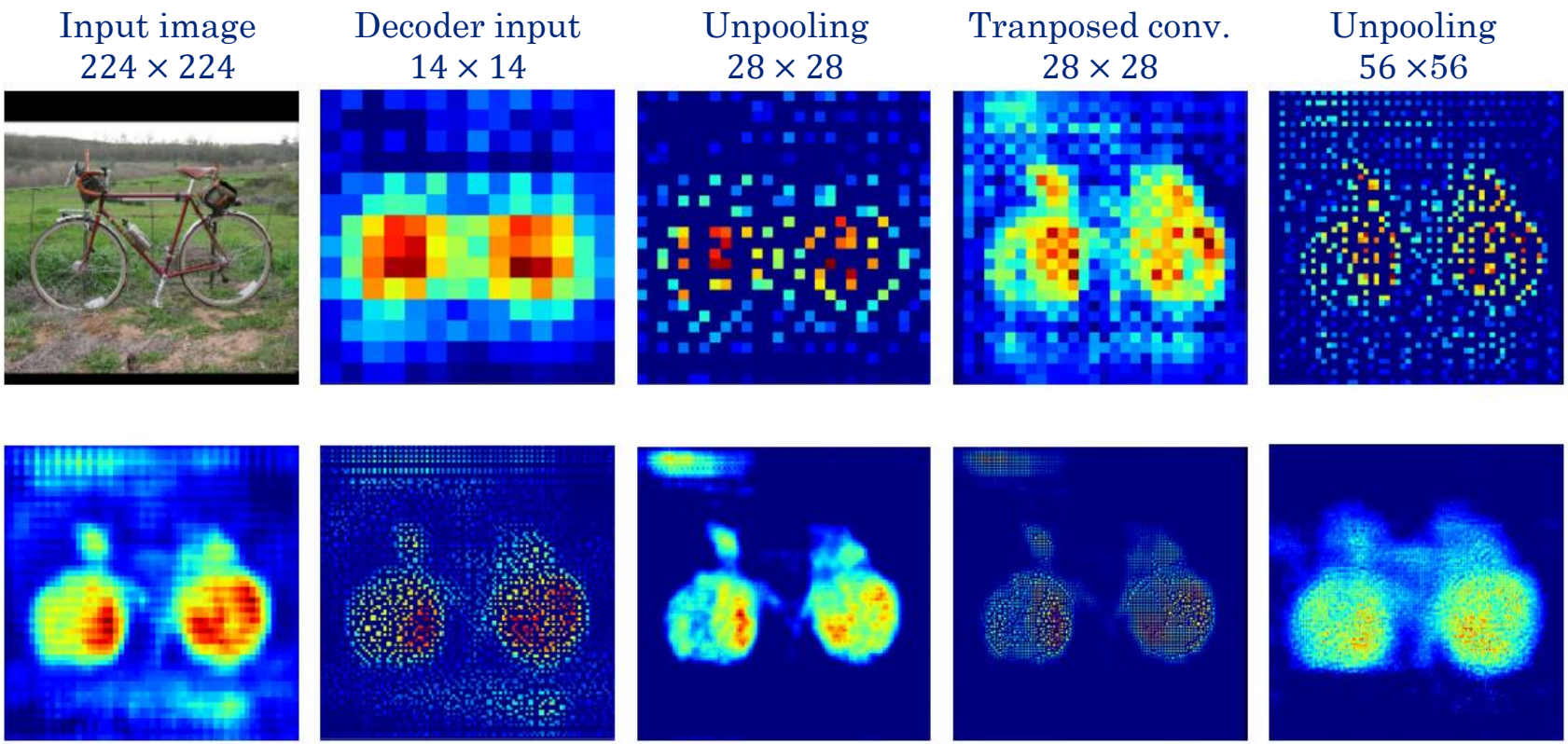
+

		0	3	6
		9	12	15
		18	21	24

=

0	0	0	1	2
0	0	3	4	5
0	2	10	10	14
6	8	19	12	15
12	14	34	21	24

Esempio decoder



Skip connection

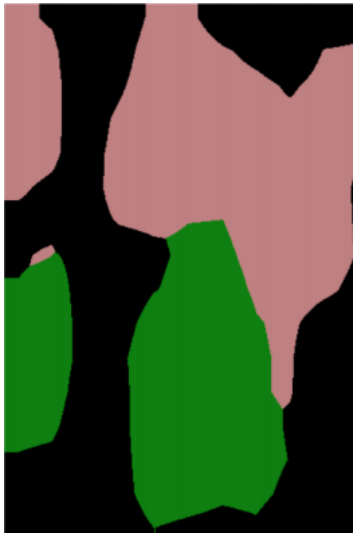
Se l'encoder e il decoder venissero messi semplicemente in sequenza si correrebbe il rischio di perdere informazioni di basso livello (dettagli).

Di conseguenza, il decoder produrrebbe un output piuttosto impreciso soprattutto nelle zone di confine tra classi diverse.

Input image

Ground truth

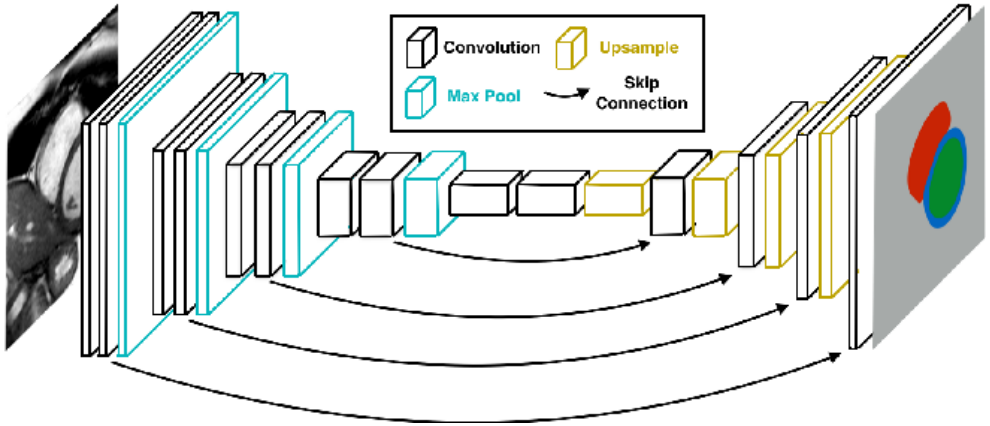
Output map

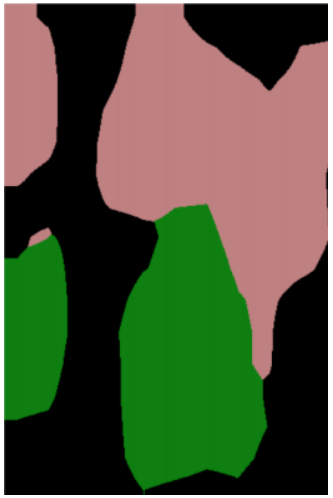


Skip connection (2)

Per ovviare a tale perdita di informazioni, si può dare la possibilità al decoder di accedere alle feature di basso livello prodotte dagli strati dell'encoder.

Ciò si può ottenere tramite le *skip connection*: gli output intermedi dell'encoder vengono aggiunti/concatenati (in posizioni opportune) agli input dei livelli intermedi del decoder.



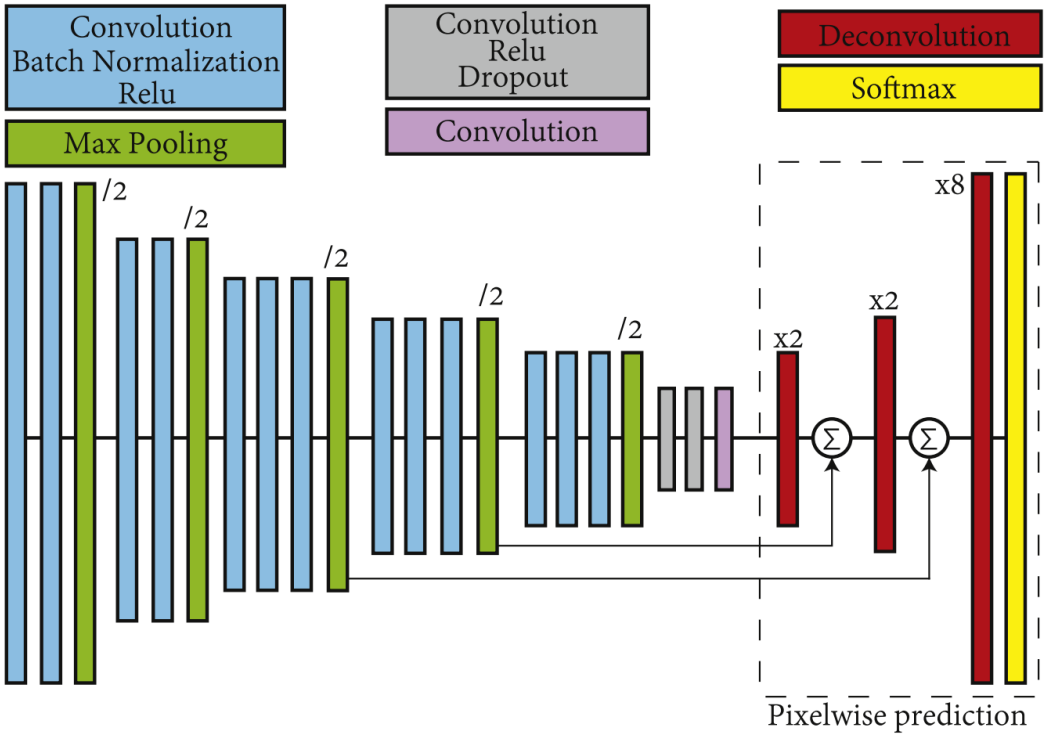


Senza skip connections

Con skip connections

Fully Convolutional Network

Long, Shelhamer e Darrell sono stati i primi nel 2015 a proporre una Fully Convolutional Network (FCN) per la segmentazione di immagini.



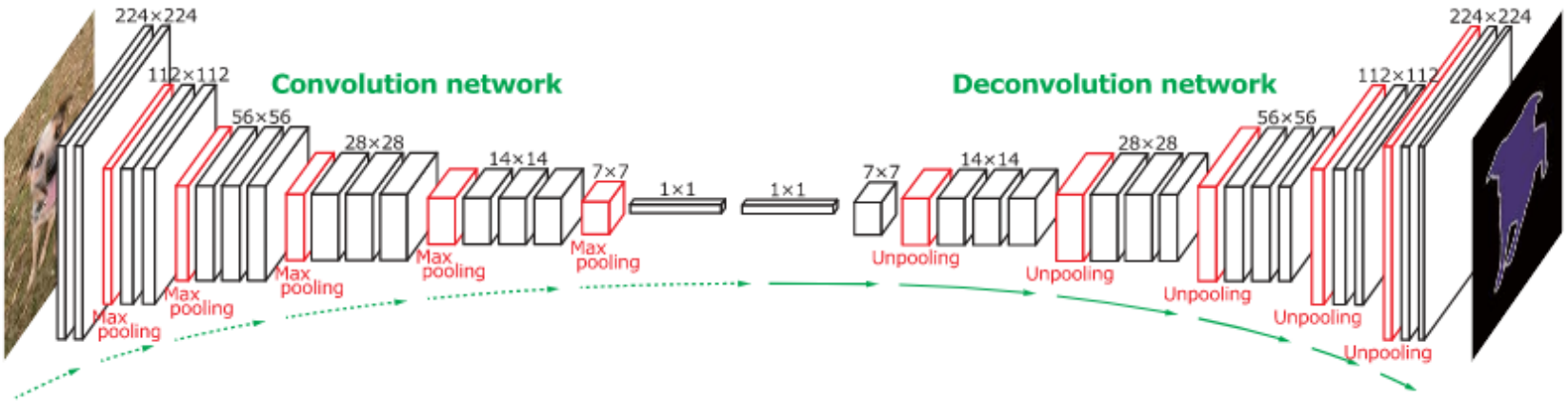
J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *proceedings of Computer Vision and Pattern Recognition (CVPR)*, June 2015.

Fully Convolutional Network (2)

- Non essendoci livelli fully-connected può essere utilizzata con immagini di qualsiasi dimensione.
- Utilizzabile con diverse architetture (*backbone*) allo stato dell'arte: VGG, AlexNet, GoogleNet.
- Addestramento effettuato tramite *fine-tuning*.

DeconvNet

Noh, Hong e Han nel 2015 hanno proposto DeconvNet, un modello composto da due parti connesse fra loro (encoder e decoder).



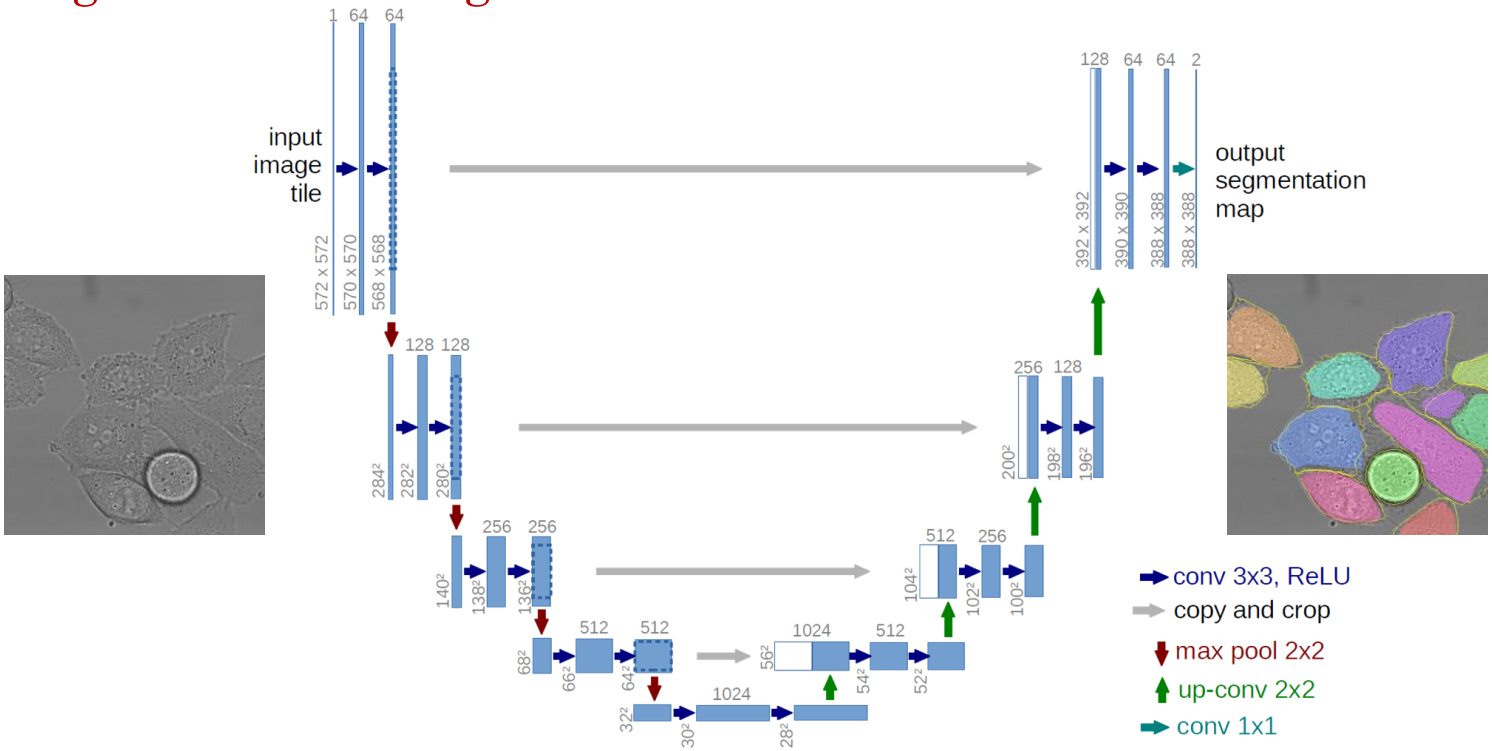
H. Noh, S. Hong, and B. Han, "Learning Deconvolution Network for Semantic Segmentation," in *proceedings of International Conference on Computer Vision (ICCV)*, December 2015.

DeconvNet (2)

- Come encoder è stata utilizzata l'architettura VGG16 ma può essere adattata anche ad altre architetture.
- L'architettura del decoder è speculare a quella dell'encoder.
- Presenza di due livelli fully-connected.
- Utilizzo di un livello di *batch normalization* dopo ogni livello di convoluzione per aumentare la velocità di apprendimento.
- Addestramento effettuato tramite *fine-tuning*.

U-Net

U-Net è stata proposta da Ronneberger, Fischer e Brox nel 2015 per segmentare immagini medicali.



O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, November 2015.

U-Net (2)

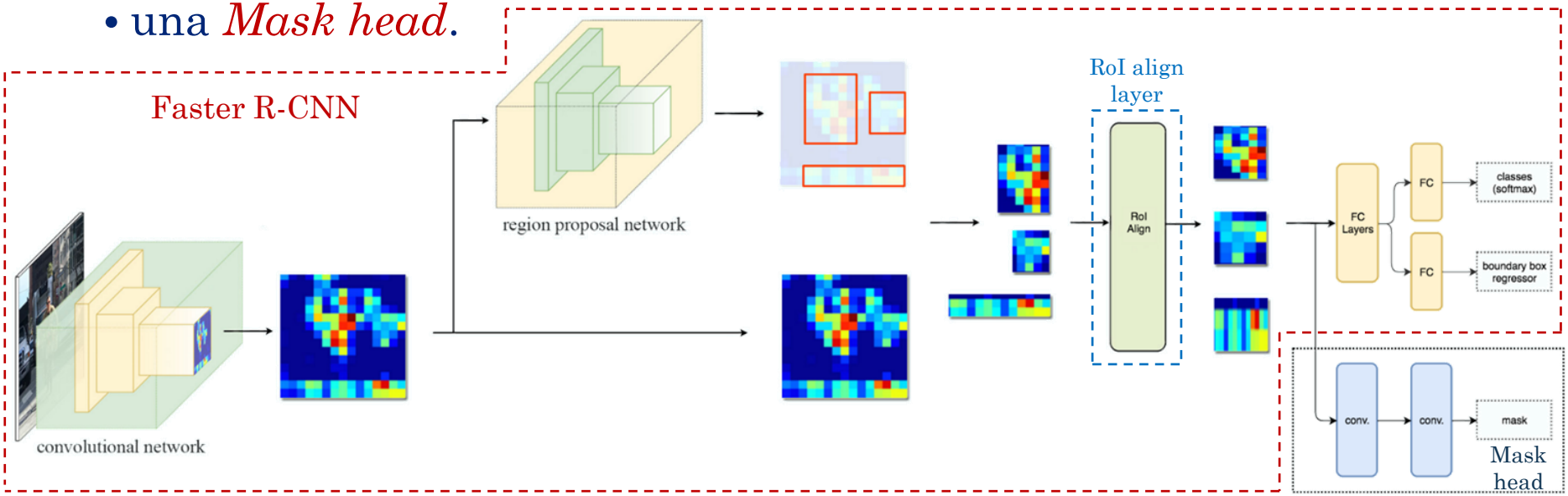
- In grado di combinare informazioni locali (encoder) e contestuali (decoder) grazie all'utilizzo di skip connections.
- Non essendoci livelli fully-connected può essere utilizzata con immagini di qualsiasi dimensione.
- Efficiente.
- Non necessità di grandi quantità di dati per essere addestrata.
- Nata per essere utilizzata in ambito medicale può essere utilizzata facilmente anche in altri contesti.
- Addestramento *end-to-end*.

Mask R-CNN

Nel 2017, He, Gkioxari, Dollar e Girshick hanno proposto Mask R-CNN per risolvere problemi di *instance segmentation*.

Mask R-CNN è composta da:

- una Faster R-CNN.
- un *RoI align layer*.
- una *Mask head*.



K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," in *proceedings of the IEEE International Conference on Computer Vision (ICCV)*, October 2017.

Mask R-CNN (2)

- La Faster R-CNN restituisce una *bounding box* e una *label* per ogni oggetto presente nell'immagine di input.
- Il RoI pooling layer della Faster R-CNN è sostituito da un RoI align layer (che approfondiremo più avanti) per meglio preservare le informazioni spaziali.
- L'output del RoI align layer è dato in input alla Mask head (che approfondiremo più avanti) che genera una maschera binaria per ogni RoI.
- Per ogni RoI utilizza una multi-task loss definita come:

$$L = L_{cls} + L_{box} + L_{mask}$$

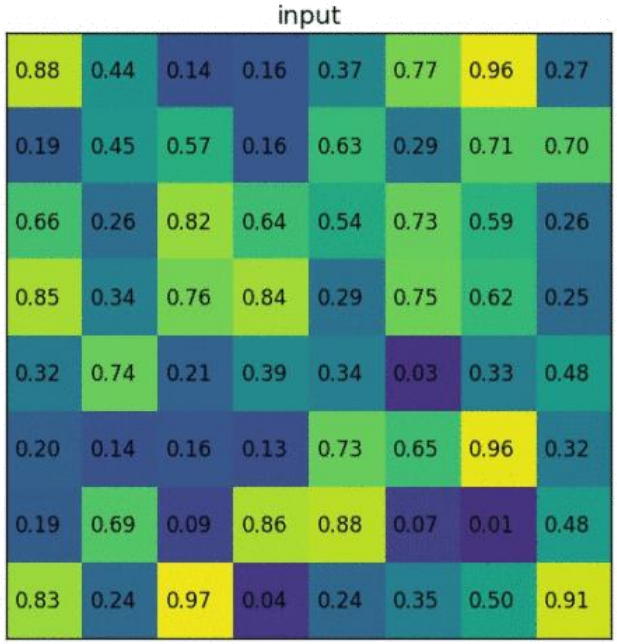
  le componenti L_{cls} e L_{box} sono identiche a quelle utilizzate nella Faster R-CNN mentre L_{mask} è la media della *binary cross-entropy* calcolata sull'intera maschera.

Mask R-CNN (3)

- Facile da addestrare.
- Efficiente.
- Performance elevate.
- Il modello è molto flessibile e facilmente adattabile alla risoluzione di altri task.

RoI align layer

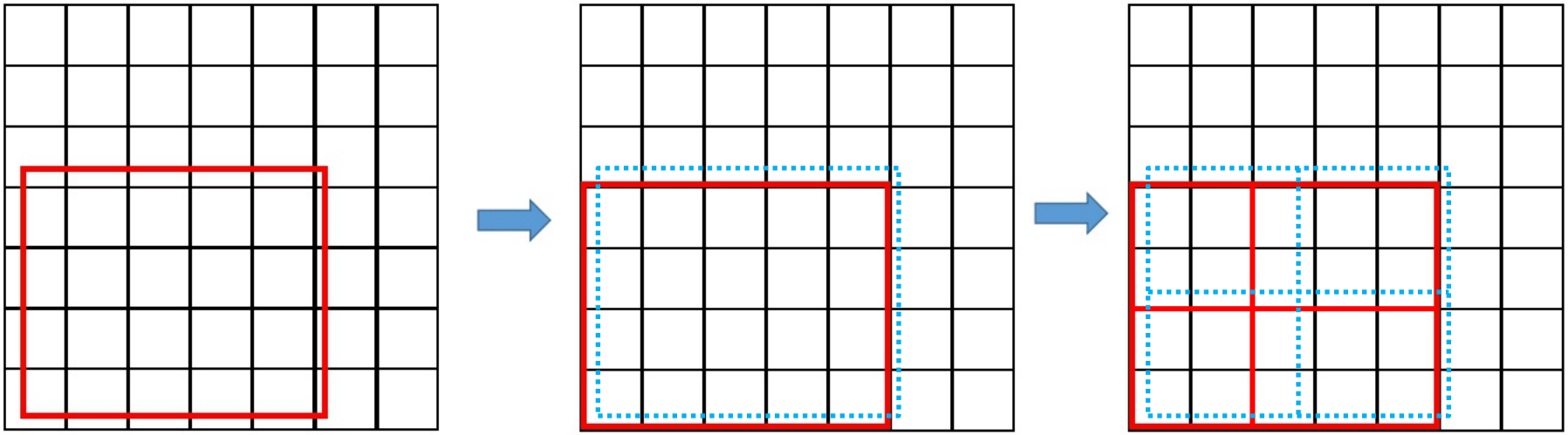
- Il RoI pooling layer restituisce un volume di output di dimensione prefissata indipendentemente dalla dimensione della regione di interesse di input.



Link

RoI align layer (2)

- Il RoI pooling layer soffre del problema del disallineamento (poco influente per problemi di *object detection*) che può essere estremamente dannoso nel caso di task di segmentazione.

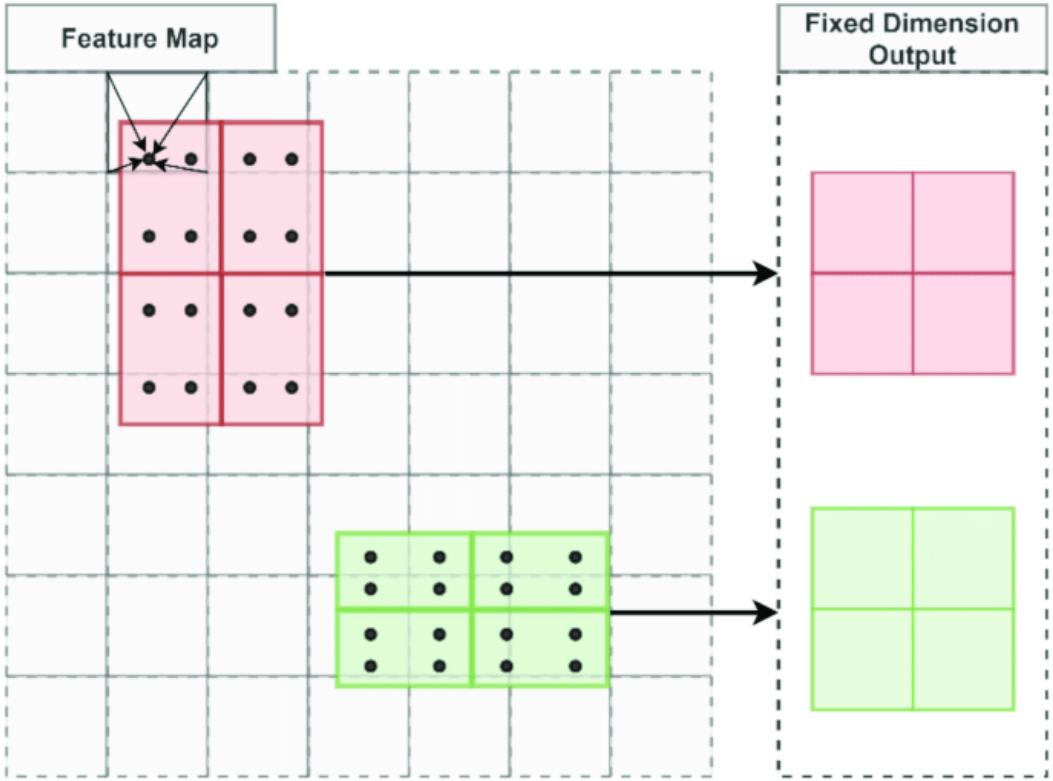


Arrotondamento di x, y, w e h

Arrotondamento della divisione in celle.

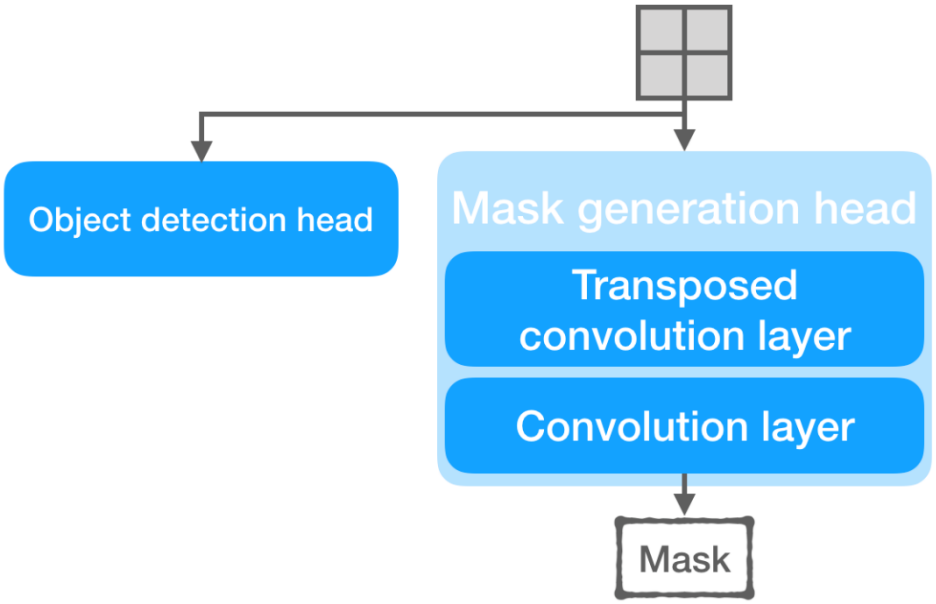
RoI align layer (3)

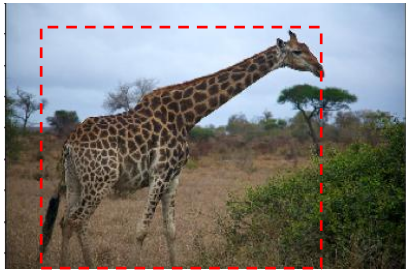
- Per evitare i due arrotondamenti causa del disallineamento, il RoI align layer utilizza l'interpolazione bilineare.

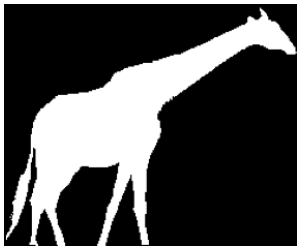


Mask head

- La Mask head è composta da due layer:
  - Transposed convolution
  - Convolution
- Restituisce una maschera binaria che si riferisce alla classe restituita dalla Faster R-CNN.

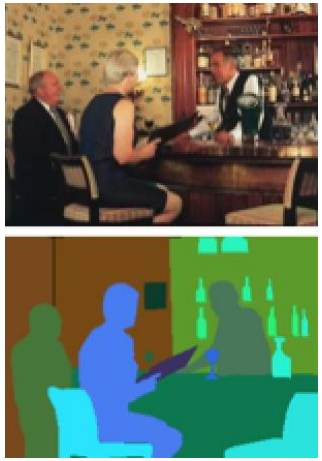






Dataset di riferimento

Dataset	# immagini	# classi	Caratteristiche
PASCAL VOC	~20K	20	Dataset storico, semplice, ideale per training rapido e sperimentazioni didattiche
Cityscapes	5K	30	Scene urbane, standard per guida autonoma
ADE20K	~27K	~150	Scene indoor/outdoor
COCO-Stuff	164K	~170	Molto complesso
Mapillary Vistas	25K	124	Scene stradali



Modelli recenti

Modello	Pro	Contro
DeepLabV3+	Ottimo per multi-scale context e contorni precisi	Computazionalmente pesante per real-time
SegFormer	Alta accuratezza, generalizzazione eccellente	Richiede GPU potenti per training
SAM	Non richiede retraining, segmenta qualsiasi oggetto	Non ottimizzato su dataset classici, ma per versatilità
PointRend	Ottimo per immagini ad alta risoluzione e video	Più complesso da integrare